GPU-NFBench: Benchmarking LLM-Assisted Triage of GPU Numerical Failures in High-Performance ML Software

Aryan Shah
Texas Academy of Mathematics and Science
University of North Texas
Denton, TX, USA
aryanshah.2431@gmail.com

Abstract—High-performance ML software increasingly depends on GPU kernels, graph compilers, low/mixed-precision arithmetic, and Python-facing accelerator libraries. When these stacks fail, issue reports about NaNs, overflow, dtype conversion, precision tolerance, incorrect outputs, compiler failures, and performance regressions often overlap. This paper presents GPU-NFBench, a human-labeled benchmark and triage framework for GPU numerical-failure reports. We construct a 1,191-issue gold benchmark from public GitHub issues across eight GPU-related repositories using a seven-class taxonomy for invalid values, overflow/underflow, precision/tolerance, dtype/casting, crash/compile, performance-only, and non-numerical reports. Weak keyword-derived labels reach only 50.5% accuracy, showing that retrieval labels are not sufficient ground truth. A deterministic voting ensemble reaches 73.8% accuracy and 0.712 macro F1; a fine-tuned FLAN-T5-base classifier reaches 80.5% held-out accuracy and 0.778 macro F1; and an LLM/deterministic agreement mode reaches 89.3% accuracy at 68.3% coverage. Cross-repository transfer remains difficult: leave-one-repository-out evaluation reaches 66.8% accuracy. A 119-row blind audit achieved 80.7% annotator agreement and Cohen's $\kappa = 0.721$, and follow-on adjudication applied 419 human updates. We release the benchmark, models, 250-row root-cause evidence-coded extension, error appendix, and reproducibility artifact with a Zenodo DOI.

Index Terms—GPU bugs, numerical bugs, benchmark construction, issue mining, LLM-assisted triage, software reliability, machine learning systems

I. Introduction

Modern AI and scientific workloads rely on GPU software stacks that combine low-level kernels, graph/compiler transformations, mixed precision, and high-level Python interfaces. Frameworks such as Triton, PyTorch, JAX, CuPy, Numba, RAPIDS, and TVM make acceleration accessible, but they also create a large surface for numerical failures: NaNs, Inf values, silent wrong answers, overflow, dtype-conversion errors, and compiler/runtime failures that appear only on specific hardware and software configurations.

Public issue trackers record these failures at scale, but they are noisy. A search for "NaN" can return true invalid-value bugs, missing-data examples, or unrelated benchmark tables; a search for "dtype" can return casting failures, unsupported features, performance complaints, or compiler crashes. GPU-NFBench addresses this by separating weak retrieval labels from human gold labels and by evaluating both full-coverage and abstaining triage systems.

The contributions are:

- We construct a 1,191-row human-labeled benchmark of GPU numerical failure reports across eight public GPU/HPC software projects.
- We define a seven-class taxonomy and preserve normalization and adjudication logs, including 419 human updates to the final benchmark.
- We evaluate weak labels, lexical baselines, linear classifiers, a fine-tuned FLAN-T5-base model, deterministic ensembles, cross-repository transfer, chronological transfer, and abstention.
- We release the data card, benchmark, model outputs, error appendix, 50-row human-adjudicated root-cause subset, 250-row evidence-coded root-cause extension, and reproducibility artifact.

II. Background and Motivation

GPU numerical bugs are difficult because symptoms and causes are often separated. A NaN can originate from overflow in a fused kernel, an unsafe dtype cast, invalid memory access, compiler lowering, or a tolerance mismatch. Issue reports rarely state the root cause directly; instead, they contain partial evidence such as a stack trace, device name, failing kernel, expected CPU result, tolerance comparison, or maintainer comment. This makes GPU numerical triage a useful HPEC problem: it combines high-performance GPU software, low/mixed precision behavior, benchmarking, reliability, and LLM-assisted analysis.

III. Benchmark Construction

The project began with a 930-issue silver seed collected from GPU-related repositories using terms such as nan, inf, overflow, dtype, precision, tolerance, incorrect, and wrong result. Candidate labels were assigned from query and text heuristics but were not treated as ground truth. We then constructed a 191-issue pilot gold benchmark

TABLE I
Expanded gold benchmark label distribution.

Primary label	Issues
precision_tolerance	368
dtype_casting	193
crash_compile	192
not_numerical_failure	147
nan_inf	143
overflow_underflow	94
performance_only	54
Total	1191

with full public issue/comment context and expanded it with 1,000 additional human-labeled issue records. The final v2 benchmark contains 1,191 issues from JAX, PyTorch, RAPIDS cuML, Triton, RAPIDS cuDF, CuPy, Numba, and Apache TVM. Table I shows the primary-label distribution.

The expanded human annotation file included several labels outside the final seven-class taxonomy, including API feature requests, compiler/code-generation descriptions, generic needs-review labels, and performance-regression labels. Rather than silently discard or recode them, we ran a deterministic taxonomy-normalization pass and preserved the full change log. In total, 180 rows were mapped into the final taxonomy, and no invalid labels remained. For example, performance-regression labels were mapped to performance_only; compiler/code-generation labels were mapped to crash_compile, performance_only, or not_numerical_failure depending on the recorded evidence; and API feature requests were mapped to not_numerical_failure. The original labels remain recoverable from the repair log, so this step is auditable.

We also ran a blind audit sampled from the expansion set. Two annotators independently labeled 119 rows using the final taxonomy, achieving 80.7% agreement and Cohen's $\kappa = 0.721$. A third adjudication pass assigned final labels to those rows, and a second 300-row adjudication pass focused on low-confidence and model-error examples. Together, these passes applied 419 human updates to the benchmark and changed 211 primary labels. The final experiments use this canonical v2 benchmark.

IV. Taxonomy

Each issue receives exactly one primary label:

- nan_inf: invalid-value failures involving NaN, Inf, or non-finite outputs.
- overflow_underflow: numerical range failures, including saturation, wraparound, or loss to zero.
- precision_tolerance: wrong-answer or tolerance failures where approximate arithmetic, accumulation order, or precision is central.
- dtype_casting: dtype promotion, conversion, unsupported dtype, or casting semantics failures.
- crash_compile: failures dominated by crashes, compiler errors, illegal memory accesses, or build/runtime exceptions.
- performance_only: reports about speed, memory use, or scheduling where no numerical correctness issue is present.
- not_numerical_failure: feature requests, API questions, documentation issues, and other non-numerical reports.

The taxonomy intentionally separates symptoms from suspected causes. For example, an issue may be labeled nan_inf as the primary symptom while secondary evidence points to dtype conversion, masking, or compiler lowering. This paper evaluates primary-label triage; secondary-cause prediction is left as a follow-up task.

V. Models and Evaluation

We evaluate models on the 1,191-row expanded gold benchmark. Metrics are accuracy and macro F1 for the seven-class task. Macro F1 is important because the dataset is imbalanced: precision/tolerance issues are the largest class, while performance-only reports are the smallest. We also evaluate a binary gate that predicts whether an issue is a numerical failure at all. The primary headline setting is stratified five-fold evaluation. We also report leave-one-repository-out evaluation, where each repository is held out as the test project and all other repositories are used for training.

The majority baseline predicts the largest class. The weak-label baseline uses the original candidate label attached during retrieval. Text baselines include BM25 nearest-neighbor retrieval, multinomial naive Bayes, TF-IDF logistic regression, TF-IDF linear SVM, and bigram TF-IDF logistic regression. These baselines are deliberately simple so that improvements can be attributed to better supervision and calibrated decision rules rather than opaque modeling choices.

We fine-tuned a standalone FLAN-T5-base sequence-to-sequence classifier on benchmark-formatted issue text from the v2 canonical benchmark. The split contains 945 training rows, 123 validation rows, and 123 held-out test rows. To reduce class imbalance, the training set was balanced by oversampling minority classes to 2,058 training examples. The selected v2 checkpoint starts from the previous expanded-gold FLAN-T5-base checkpoint and is retrained for one epoch on the v2 labels. It predicts one of the seven primary taxonomy labels directly from issue text.

The best full-coverage deterministic system is a voting ensemble over the strongest lexical and linear models. For selective triage, the ensemble can abstain unless at least k voters agree. This setting matches realistic engineering use: a triage assistant should confidently label straightforward issues and flag ambiguous reports for human review.

TABLE II
Expanded gold benchmark full-coverage results.

Model	Accuracy	Macro F1
Majority baseline	0.309	0.067
Candidate weak label	0.505	0.426
BM25 nearest neighbor	0.571	0.534
Naive Bayes	0.610	0.501
TF-IDF logistic regression	0.715	0.688
TF-IDF linear SVM	0.732	0.703
Bigram TF-IDF logistic	0.724	0.694
Voting ensemble	0.738	0.712

TABLE III
Leave-one-repository-out evaluation on expanded gold.

Model	Accuracy	Macro F1
Candidate weak label	0.505	0.426
BM25 nearest neighbor	0.493	0.478
Naive Bayes	0.550	0.475
TF-IDF logistic regression	0.653	0.630
TF-IDF linear SVM	0.668	0.648
Bigram TF-IDF logistic	0.663	0.633
Voting ensemble	0.668	0.647

We also evaluate an LLM-assisted agreement mode. This mode answers only when the deterministic ensemble and standalone LLM predict the same label; otherwise it abstains. This is stricter than majority voting and is intended for high-confidence triage queues where false labels are more costly than deferred labels.

VI. Results

A. Full-Coverage Classification

Table II reports full-coverage performance. Weak candidate labels reach only 50.5% accuracy, confirming that keyword retrieval labels are not reliable gold labels. Linear text models perform substantially better with the expanded human-labeled set. The full-coverage voting ensemble reaches 73.8% accuracy and 0.712 macro F1.

B. Cross-Repository Generalization

Table III reports leave-one-repository-out results. This setting is harder because model vocabularies, issue templates, and library-specific concepts shift across projects. The best deterministic models in this setting reach 66.8% accuracy, with TF-IDF linear SVM reaching 0.648 macro F1 and the voting ensemble reaching 0.647 macro F1. The result is substantially above the weak-label baseline but lower than shuffled evaluation, indicating that cross-project transfer remains a central challenge for GPU numerical-failure triage.

Table IV summarizes the weakest transfer repositories. CuPy and Numba are the hardest held-out projects: CuPy mixes dtype/API compatibility with numerical symptoms, while Numba contains CUDA, LLVM, build, and runtime language that blurs crash_compile and not_numerical_failure. This supports cross-repository transfer as a real benchmark challenge rather than a simple shuffled-text classification task.

TABLE IV
Weakest leave-one-repository-out transfer cases.

Held-out repo	Issues	Best acc.	Ensemble acc.
numba/numba	65	0.492	0.462
cupy/cupy	82	0.500	0.439
rapidsai/cudf	132	0.583	0.545

TABLE V
Chronological 80/20 evaluation on expanded gold.

Model	Accuracy	Macro F1
Candidate weak label	0.469	0.325
Naive Bayes	0.598	0.475
TF-IDF logistic regression	0.711	0.690
TF-IDF linear SVM	0.732	0.697
Bigram TF-IDF logistic	0.766	0.738
Voting ensemble	0.745	0.722

C. Chronological Generalization

We also evaluate a future-facing chronological split. Issues are sorted by creation time; the older 80% form the training set and the newer 20% form the test set. The split trains on 952 issues through December 17, 2025 and tests on 239 newer issues. Table V shows that bigram TF-IDF logistic regression reaches 76.6% accuracy and 0.738 macro F1, while the voting ensemble reaches 74.5% accuracy and 0.722 macro F1. These results suggest that the benchmark supports triage of later issue reports, but also that the simpler linear model transfers better than the ensemble under temporal shift.

D. Standalone LLM Performance

Table VI compares the standalone LLM with the best deterministic ensemble. The v2 standalone FLAN-T5-base model reaches 80.5% held-out test accuracy and 0.778 macro F1, outperforming the deterministic ensemble on the same held-out LLM test split. This changes the practical conclusion from the earlier expanded-gold version: after adjudication repair, the standalone LLM is not only a simpler deployable artifact, but also the strongest full-coverage triage model in the final held-out evaluation. As an additional local zero-shot comparison, we evaluated Ollama llama3.2:3b on the same 123 held-out rows. It reaches only 31.7% accuracy and 0.322 macro F1, showing that task-specific fine-tuning is important. A cloud-backed modern model was not used for the final comparison because exporting held-out benchmark prompts to an external service was blocked in the local environment.

E. Selective Triage

Table VII shows the accuracy/coverage tradeoff when the ensemble abstains unless enough voters agree. With at least three agreeing voters, the system labels 92.6% of issues at 76.9% accuracy. With at least four agreeing voters,

TABLE VI
Standalone LLM and deterministic comparison on v2 labels.

Model	Accuracy	Macro F1
Local Llama 3.2 3B zero-shot	0.317	0.322
TF-IDF linear SVM on LLM test split	0.691	0.669
Voting ensemble on LLM test split	0.683	0.654
FLAN-T5-base standalone LLM	0.805	0.778
Binary numerical gate, 5-fold	0.892	0.796

TABLE VII
Selective prediction with abstention.

Agreement rule	Coverage	Accuracy	Macro F1
Vote ≥ 2	0.999	0.739	0.713
Vote ≥ 3	0.926	0.769	0.744
Vote ≥ 4	0.735	0.842	0.803
Vote ≥ 5	0.301	0.939	0.733
LLM/ensemble agreement	0.683	0.893	0.844

it labels 73.5% of issues at 84.2% accuracy. With unanimous or near-unanimous agreement, it reaches 93.9% accuracy on 30.1% of issues. The LLM/deterministic agreement mode provides a complementary high-confidence setting: it answers 68.3% of held-out LLM-test examples at 89.3% accuracy and 0.844 macro F1.

F. Error Analysis

The largest error families are semantically plausible boundary cases. The most common confusion is dtype_casting predicted as precision_tolerance (28 errors), followed by nan_inf predicted as precision_tolerance (22 errors), precision_tolerance predicted as dtype_casting (21 errors), and crash_compile predicted as precision_tolerance (20 errors). These errors reflect the structure of GPU numerical reports: dtype changes, compiler failures, overflow, and NaN symptoms are often reported through failed numerical tolerance tests. The next largest errors involve not_numerical_failure issues predicted as crash or dtype failures, usually because issue text contains API type errors or installation stack traces even when the report is not a numerical defect. This analysis supports two follow-up tasks: multi-label secondary-cause prediction and explicit separation of symptom labels from root-cause labels.

The artifact includes a 12-case error appendix with issue snippets and boundary explanations. Examples include dtype reports predicted as precision/tolerance because the failure appears as a numerical mismatch, NaN/Inf reports embedded in broader tolerance failures, and non-bug support requests predicted as crash or dtype failures because they contain stack-trace or type-language cues.

The artifact includes 12 concrete error cases with issue snippets and boundary explanations. It also contains linked-fix evidence for future root-cause work: 105 pilot issues contain at least one fix or root-cause signal, including 39 explicit pull-request URLs, and a public diff fetch retrieved 488 changed-file rows. We additionally provide a 50-row human-adjudicated root-cause subset and a 250-row evidence-coded extension. The extension contains the 50 human-adjudicated rows, 55 linked-fix evidence-coded rows, and 145 issue-text evidence-coded rows; it is released as root-cause supervision for future work, not as 250 human-adjudicated labels.

VII. Discussion

The expanded gold set changes the practical conclusion. On the original 191-issue pilot, full-coverage classification remained near 54%, and the most useful mode was abstention. With 1,191 human-labeled examples, full-coverage triage becomes viable: the deterministic ensemble reaches 73.8% accuracy, the standalone LLM reaches 80.5% held-out accuracy, and chronological evaluation reaches 76.6%. The standalone LLM is the simplest deployable full-coverage artifact, while the deterministic ensemble remains useful as a transparent baseline and confidence mechanism. For HPEC-style workflows, GPU-NFBench can help maintainers prioritize numerical defects, help researchers sample failure clusters for root-cause analysis, and provide a supervised evaluation set for future debugging agents before patch localization or reproducer generation.

VIII. Threats to Validity

The benchmark relies on human labels and deterministic normalization. Although all repairs are logged and the audit shows substantial agreement, the full 1,191-row benchmark is not double-adjudicated. The dataset is drawn from public GitHub issues and may underrepresent private industrial bugs, closed-source compiler failures, or hardware-specific failures not reported publicly. Repository sizes are uneven, so cross-project transfer should be interpreted with care. Finally, many issue reports lack minimal reproducers or confirmed fixes; the primary task classifies reports, while linked-fix root-cause prediction remains future work.

IX. Conclusion

This paper presents GPU-NFBench, a 1,191-row human-labeled benchmark for GPU numerical failure reports in high-performance ML software. Replacing weak keyword labels with human gold labels makes reliable triage possible: a standalone FLAN-T5-base model reaches 80.5% held-out accuracy, a deterministic ensemble reaches 73.8%, chronological evaluation reaches 76.6%, and LLM/deterministic agreement reaches 89.3% accuracy at 68.3% coverage. Leave-one-repository-out accuracy is 66.8%, showing that cross-project transfer remains difficult. The artifact provides a foundation for GPU reliability triage, benchmark construction, and root-cause prediction.

Artifact Availability

The artifact contains the processed benchmark, expanded gold labels, taxonomy-normalization report, model-training scripts, fine-tuning files, evaluation tables, generated reports, qualitative examples, linked-fix evidence, a data card, completed blind audit files, v2 adjudication files, LLM fine-tuning files, the 50-row human-adjudicated root-cause subset, the 250-row evidence-coded root-cause extension, cross-repository weakness analysis, local LLM baseline outputs, a 12-case error appendix, a reproducibility checklist, and an external-repository candidate pool for future benchmark expansion. The data card documents intended use, non-use cases, label provenance, and known limitations. The artifact is released at https://github.com/jubs-2431/gpu-nfbench/releases/tag/v1.0-conference. The archived Zenodo record is available at https://zenodo.org/records/20242157 with DOI https://doi.org/10.5281/zenodo.20242157.

References

[1] P. Tillet, H. Kung, and D. Cox, "Triton: An intermediate language and compiler for tiled neural network computations," in Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages, 2019.

[2] R. Okuta, Y. Unno, D. Nishino, S. Hido, and C. Loomis, "CuPy: A NumPy-compatible library for NVIDIA GPU calculations," in Proceedings of Workshop on Machine Learning Systems at NeurIPS, 2017.

[3] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Advances in Neural Information Processing Systems, 2019.

[4] J. Bradbury et al., "JAX: Composable transformations of Python+NumPy programs," 2018. [Online]. Available: https://github.com/jax-ml/jax

[5] S. K. Lam, A. Pitrou, and S. Seibert, "Numba: A LLVM-based Python JIT compiler," in Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC, 2015.

[6] RAPIDS Development Team, "RAPIDS: Collection of GPU accelerated data science libraries," 2024. [Online]. Available: https://rapids.ai/

[7] T. Chen et al., "TVM: An automated end-to-end optimizing compiler for deep learning," in 13th USENIX Symposium on Operating Systems Design and Implementation, 2018.

[8] C. Raffel et al., "Exploring the limits of transfer learning with a unified text-to-text transformer," Journal of Machine Learning Research, vol. 21, no. 140, pp. 1–67, 2020.

[9] H. W. Chung et al., "Scaling instruction-finetuned language models," J. Mach. Learn. Res., vol. 25, no. 70, pp. 1–53, 2024.